

Analysis of Simplex Methods

June 10, 2026

1 Python codes

1.1 Code for find_pivot_largest_subscript

```
1 def find_pivot_largest_subscript(tableau):
2     m = len(tableau[0]) - 2          #columns minus the RHS
3     n = len(tableau) - 1            #rows
4
5     k = m
6     while tableau[n][k] <= 0:
7         k -= 1
8     final_s = k
9
10    theta_star = np.inf
11    for i in reversed(range(n)):
12        if tableau[i][final_s] > 0:
13            ratio = tableau[i][-1] / tableau[i][final_s]
14            if ratio < theta_star:
15                theta_star = ratio
16            r = i
17    final_r = r
18
19    return final_r, final_s
```

1.2 Code for find_pivot_largest_change

```
1 def find_pivot_largest_change(tableau):
2     m = len(tableau[0]) - 2
3     n = len(tableau) - 1
4     change = -np.inf
5     final_r = None
6     final_s = None
7
8     for s in range(m):
9         theta_star = np.inf
10        r = None
11        if tableau[-1][s] > 0:
12
13            for i in range(n):
14                if tableau[i][s] > 0:
15                    ratio = tableau[i][-1] / tableau[i][s]
16                    if ratio < theta_star:
17                        theta_star = ratio
18                        r = i
```

```

19
20         if theta_star == np.inf:
21             continue
22         if r is None:
23             continue
24
25         new_change = tableau[-1][s] * theta_star
26
27         if new_change > change:
28             final_s = s
29             change = new_change
30             final_r = r
31
32     if final_r is None or final_s is None:
33         return -1, -1
34
35     return final_r, final_s

```

2 Complexity analysis

2.1 Complexity of find_pivot_largest_subscript

Complexity of the previous code in section 1.1.

- Lines 2-to-5 take $\mathcal{O}(1)$.
- Lines 6 and 7:
 - Each time line 7 is repeated, $\mathcal{O}(1)$
 - Can be repeated at most m times
 - So overall, lines 6 and 7 $\mathcal{O}(m)$
- Lines 8 and 10 take $\mathcal{O}(1)$
- Lines 11-to-16:
 - Lines 12-to-16 take $\mathcal{O}(1)$
 - Can repeat a maximum of n times
 - So overall, lines 11-to-16 $\mathcal{O}(n)$
- Lines 17 and 19 take $\mathcal{O}(1)$

Overall, we have a complexity of:

$$\mathcal{O}(1) + \mathcal{O}(m) + \mathcal{O}(1) + \mathcal{O}(n) + \mathcal{O}(1) = \mathcal{O}(m + n). \quad (2.1)$$

2.2 Complexity of find_pivot_largest_change

Complexity of the previous code in section 1.2.

- Lines 2-to-6 take $\mathcal{O}(1)$.
- Each time the block of lines 8-to-30 is repeated:
 - Lines 9-to-11 $\mathcal{O}(1)$.
 - For lines 13-to-18
 - * Each line is $\mathcal{O}(1)$

- * A maximum of n times run, so overall, $\mathcal{O}(n)$
- Lines 20-to-30 are all $\mathcal{O}(1)$, so overall $\mathcal{O}(1)$
- Can be run a maximum of m times
- Overall, lines 8-to-30 take $\mathcal{O}(m)(\mathcal{O}(1) + \mathcal{O}(n) + \mathcal{O}(1))$
- Lines 32-to-35 all take $\mathcal{O}(1)$, so overall $\mathcal{O}(1)$

Overall, we have a complexity of:

$$\mathcal{O}(1) + \mathcal{O}(m)(\mathcal{O}(1) + \mathcal{O}(n) + \mathcal{O}(1)) + \mathcal{O}(1) = \mathcal{O}(mn). \quad (2.2)$$

3 Excel analysis

3.1 Best Fit Coefficients

A table showing the best fit coefficients p_0 and p_1 of the equation $\log_{10}(t) = p_0 \log_{10}(n) + p_1$ for the average (Avg) and worst (Wst) values to 4 d.p. for Bland's (B), Largest Subscript (LS) and Largest Change (LC) Rules.

Pivoting Rule	B - Avg	LS - Avg	LC - Avg	B - Wst	LS - Wst	LC - Wst
p_0	1.5335	1.4964	1.8134	1.5096	1.4604	1.8896
p_1	-5.1712	-5.0956	-5.4659	-4.6490	-4.6247	-5.3163

3.2 Figures

Two figures highlighting the relationship between run timings, and instance sizes

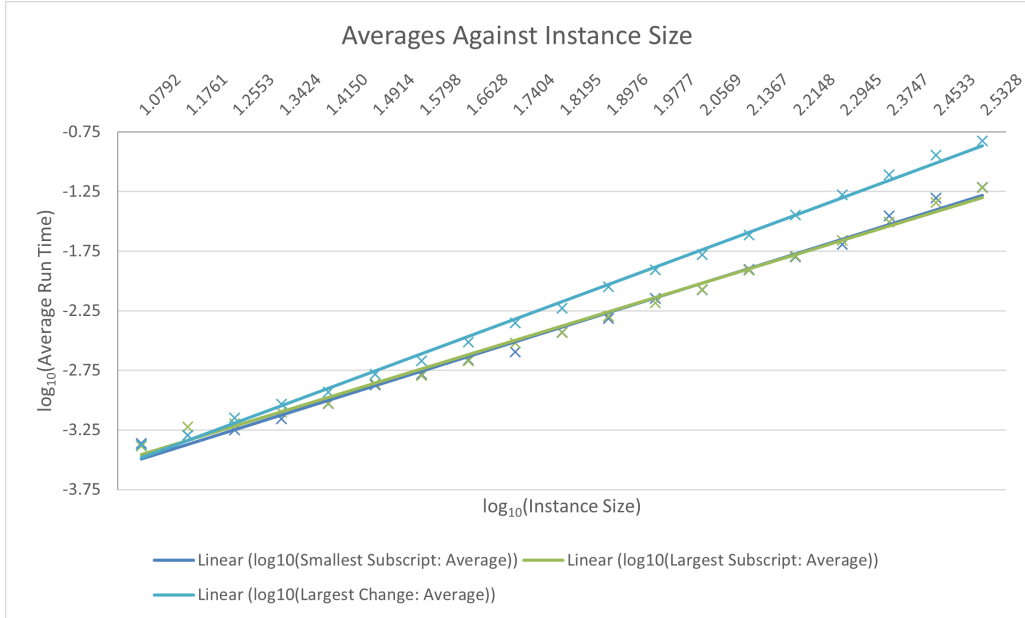


Figure 1: A figure to show the $\log_{10}$ of the average timings of each pivoting rule against the $\log_{10}$ of the instance size with the average gradient plotted.

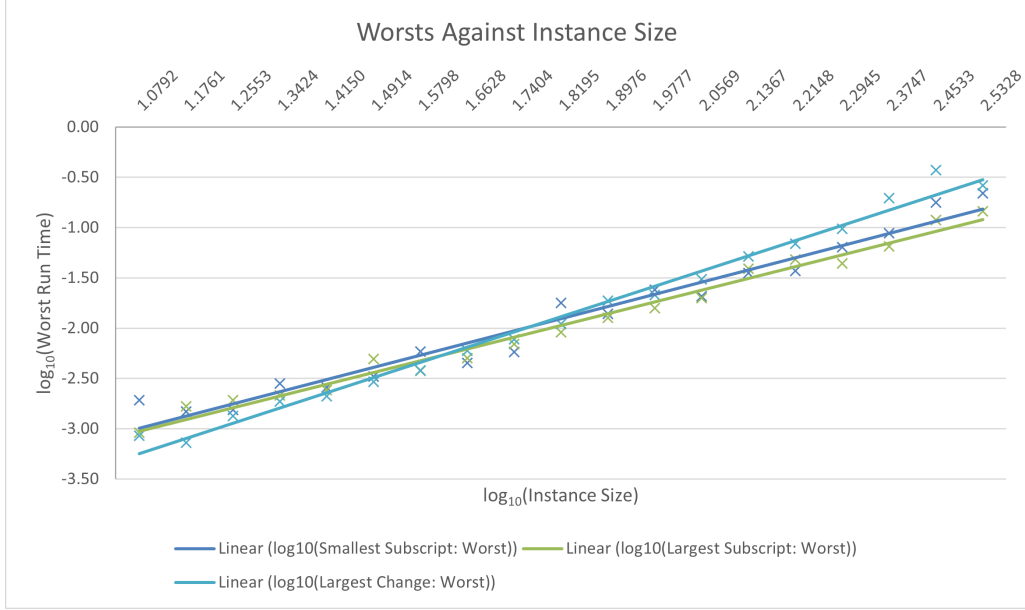


Figure 2: A figure to show the $\log_{10}$ of the worst timings of each pivoting rule against the $\log_{10}$ of the instance size with the average gradient plotted.

4 Efficiency

The complexity of the Largest Subscript Rule is $\mathcal{O}(m+n)$, whereas the complexity of the Largest Change Rule is $\mathcal{O}(mn)$. Bland's Rule (also known as the smallest subscript rule) will have the same complexity as the largest subscript rule. The only difference between the two methods is one starts at index 0 and moves forward, whilst the other starts at the end and moves backwards. So the complexity of Bland's Rule is $\mathcal{O}(m+n)$. In general mn will be larger than $m+n$ and so the Largest Change Rule has a higher complexity, therefore the Largest Subscript Rule and Bland's Rule are more efficient.

$$\log_{10}(t) = (p_0 \log_{10}(n) + p_1) \iff t = 10^{(p_0 \log_{10}(n) + p_1)} \quad (4.1)$$

The higher the efficiency, the smaller the run timings, which consequently means the smaller the values of p_0 and p_1 - as shown by the equation above. A smaller (more negative) value of p_0 appears as a shallower slope, and a smaller value of p_1 appears as a lower intercept. From the Average Timings graph (1) (it is more accurate than the Worst Timings), the pivot rule with the shallowest slope is the Largest Subscript Rule, and the one with the lowest intercept is the Largest Change Rule (though all three are *very* close). The table agrees with this.

Overall, the Largest Subscript Rule seems to be the most efficient, from both the complexity calculations and the data itself.